

JEGYZŐKÖNYV

Modern adatbázis rendszerek MSc
Oracle ORDBMS – XMLType, XMLTABLE, UDT

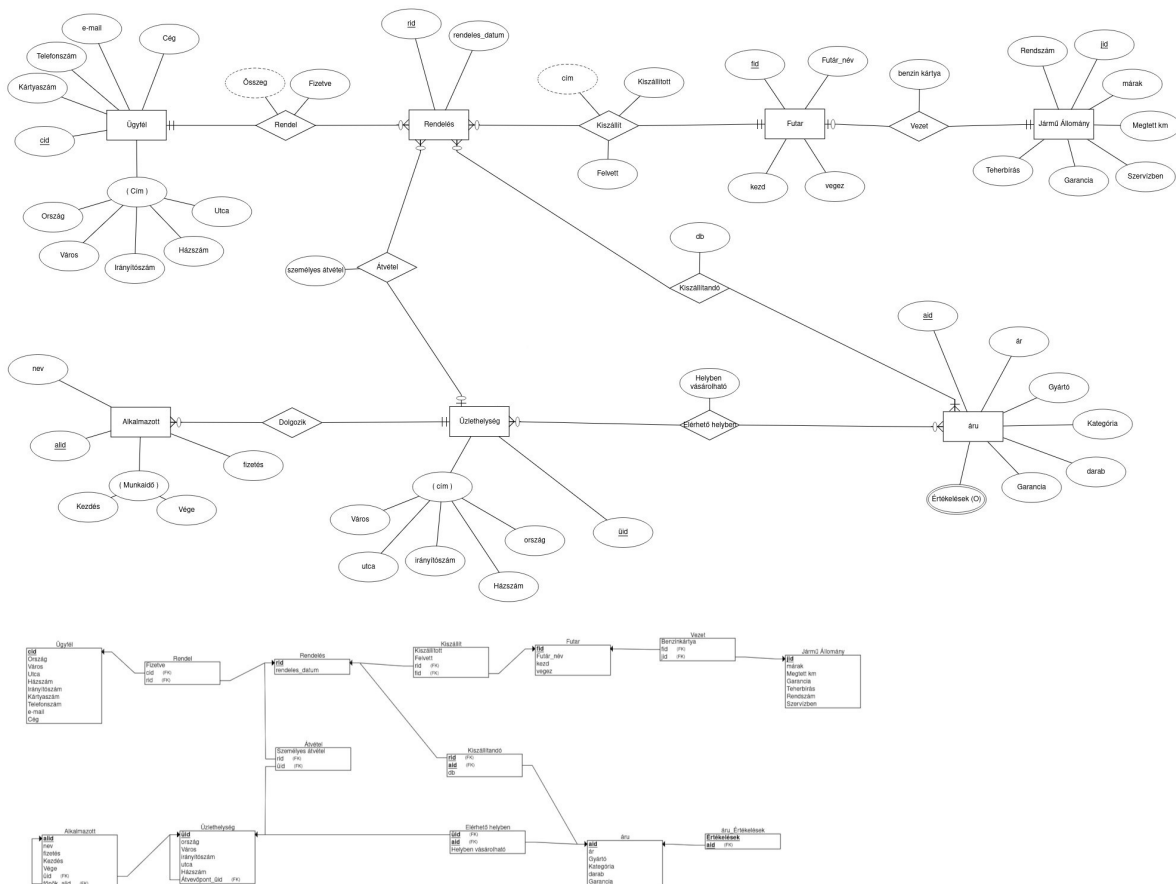
Készítete: Bán Péter
Neptunkód: Y4F3Z7
Dátum: 2026. május 13.

1. ORDBMS	3
2. A prezentáció elméleti anyagának kidolgozása	4
2.1 XMLType – Tervezés és cél	4
2.2 XMLTABLE – Az átalakítás logikája	5
2.3 UDT – Felhasználói típusok	5
2.4 VARRAY – Többszörös elemek kezelése	6
3. A gyakorlati feladatok kidolgozása	6
3.1 0. Feladat – Fejlesztőkörnyezet és XML feltöltése	7
3.2 1. Feladat – XMLTABLE lekérdezések	7
3.3 2–3. Feladat – UDT létrehozása és Object Table feltöltése	7
3.4 4–5. Feladat – Lekérdezések, módosítás, törlés	8
3.5 6. Feladat – VARRAY kollekció	8
4. Összefoglalás	9

1. ORDBMS

Az ORDBMS (Object-Relational Database Management System) egy hibrid adatbázis architektúra, amely ötvözi a hagyományos relációs modell megbízhatóságát az objektumorientált programozás rugalmasságával. A tisztán relációs rendszerekkel ellentétben az ORDBMS lehetővé teszi, hogy a fejlesztő saját adattípusokat (UDT – User-Defined Type) definiáljon, ezekhez metódusokat rendeljen, és az így létrehozott típusokat táblákhoz használja. Ez azt jelenti, hogy az adatbázis nemcsak adatot tárol, hanem az adathoz tartozó viselkedést is. Az Oracle Database az egyik legismertebb ORDBMS implementáció, amely ezt a megközelítést az XMLType, az Object Table és a VARRAY kollekciók formájában is megtestesíti.

A relációs modellhez képest az ORDBMS legnagyobb előnye az összetett, hierarchikus struktúrák leképezése. Egy egyszerű relációs sémában például egy étterem címét külön táblában kellene tárolni, és JOIN művelettel kellene visszanyerni még az ORDBMS-ben ez egy beágyazott objektum (pl. `cim_t`), amely közvetlenül az étterem-objektum részeként érhető el pont-notációval (e.g. `cim.varos`). Ezáltal a kód olvashatóbb lesz, a lekérdezések egyszerűbbek, és az adatmodell közelebb kerül a valóság struktúrájához. Az ORDBMS ugyanakkor megőrzi a relációs világ erősségeit az ACID tranzakciókat, az SQL szabványt és az érett indexelési technikákat, így nem egy teljesen új paradigmát kínál, hanem egy evolúciós lépést a relációs adatbázisok fejlődésében.



XML részlet:

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<?xml-stylesheet type="text/xsl" href="bolt.xsl"?><bolt>
  <ugyfel cid="c1">
    <nev>Kiss Péter</nev>
    <email>kiss.peter@gmail.hu</email>
    <telefonszam>06301234567</telefonszam>
    <kartyaszam>1234567812345678</kartyaszam>
    <ceg>>false</ceg>
    <cim>
      <orszag>Magyarország</orszag>
      <varos>Budapest</varos>
      <iranyitoszam>1111</iranyitoszam>
      <utca>Fő utca</utca>
      <hazszam>12</hazszam>
    </cim>
  </ugyfel>
  <aru aid="a10">
    <nev>Laptop</nev>
    <gyarto>Lenovo</gyarto>
    <kategoria>Elektronika</kategoria>
    <raktarkeszlet>5</raktarkeszlet>
    <garancia_honap>24</garancia_honap>
    <ertekelesek>
      <ertek>5</ertek>
    </ertekelesek>
  </aru>
</bolt>
```

2. A prezentáció elméleti anyagának kidolgozása

A 8. gyakorlat prezentációja az Oracle ORDBMS architektúra főbb elemeit mutatja be: az XMLType adattípust, az XMLTABLE függvényt, a felhasználói típusokat (UDT), az Object Table-t, valamint a VARRAY kollekciókat. Az alábbiakban a prezentáció minden blokkjának elméleti háttérét és tervezési logikáját ismertetem.

2.1 XMLType – Tervezés és cél

Az Oracle XMLType adattípus bevezetésének célja az volt, hogy a relációs adatbázis natívan képes legyen XML dokumentumokat tárolni és kezelni nem egyszerű szöveggént, hanem értelmezett, lekérdezhető struktúráként. A prezentáció ezt a különbséget emeli ki: míg egy VARCHAR2 oszlopban tárolt XML csak szöveg, addig az XMLType oszlopban az adatbázis érti a hierarchiát.

A tervezési döntés mögött az áll, hogy az XML dokumentumok struktúrája dinamikus és sokrétű lehet, ezért nem mindig praktikus előre definiált relációs sémába kényszeríteni. Az XMLType lehetővé teszi az XPath alapú lekérdezéseket, XSD sémaválidációt, és XML csomópontokon való indexelést.

Miért jobb mint VARCHAR2?

- XPath lekérdezések közvetlenül SQL-ből hívhatók
- Az adatbázis sémavalidációt végezhet XSD alapján
- Indexelés XML csomópontokon lehetséges
- Natív SQL integráció: az XML adatok JOIN-olhatók relációs táblákkal

Az XMLType tárolási módjai

Az Oracle az XMLType adatokat háromféle módon tudja fizikailag tárolni. Az objektum-relációs tárolás (OBJECT-RELATIONAL) során az XML struktúrát az adatbázis előre szétbontja belső táblákba, ami gyors XPath-lekérdezést tesz lehetővé, de az adatok módosítása nehezebb. A CLOB-alapú tárolás az egész XML dokumentumot szöveggént őrzi meg, rugalmas, de XPath-on való lekérdezéshez az egész dokumentumot be kell olvasni. A bináris XML tárolás (Binary XML) a legmodernebb megközelítés: az Oracle belső, optimalizált tokenizált formátumban tárolja az XML-t, ami mind a lekérdezést, mind a tárolási hatékonyságot javítja.

A mi feladatunkban az egyszerű CLOB-alapú megközelítést alkalmaztuk, ami kisebb adatmennyiség esetén teljesen megfelelő, és az APEX drag & drop feltöltési funkcióval is kompatibilis.

2.2 XMLTABLE – Az átalakítás logikája

Az XMLTABLE függvény az XML dokumentum csomópontjait sorokká és oszlopokká alakítja. Ez az úgynevezett "shredding" technika, amelynek logikája a következő lépésekből áll:

- Megadjuk az XPath gyökérútvonalat, ez határozza meg, melyik XML elem lesz egy sor
- A PASSING kulcsszóval adjuk meg, melyik XMLType oszlopot dolgozza fel
- A COLUMNS blokkban definiáljuk az egyes oszlopokat és az XPath-jaikat
- Az @ prefix attribútumokra mutat (pl. @ekod), míg a sima név gyerekeleme (pl. nev)
- A beágyazott elemekre a cim/varos szintaxis vonatkozik

A tervezési szempont: az XMLTABLE virtuális táblát hoz létre, amelyre ezután normál SQL WHERE, GROUP BY, ORDER BY feltételek alkalmazhatók.

XPath szintaxis összefoglalása

Az XMLTABLE COLUMNS blokkjában az XPath kifejezések határozzák meg, hogyan kerülnek kinyerésre az egyes értékek. Az @attribútum_neve szintaxis az XML elem attribútumát olvassa ki (pl. @ekod az ekod='e1' értékét adja vissza). A gyerekelem_neve szintaxis a közvetlen gyerekelemet olvassa (pl. nev a <nev>Trófea</nev> szövegét). A szülő/gyerek szintaxis a hierarchián belüli mély elérést teszi lehetővé (pl. cim/varos a <cim><varos>Budapest</varos></cim> struktúrából a Budapest értéket nyeri ki). Ha az XPath nem talál egyező csomópontot, az adott oszlop NULL értéket kap – ez fontos szempont, ha az XML dokumentum nem teljesen egységes felépítésű.

Az XMLTABLE egyik különleges erőssége, hogy ugyanazon XML dokumentumból egyszerre több XMLTABLE hívás is futtatható a FROM záradékban. Ezáltal JOIN-szerű összekapcsolás valósítható meg különböző XML elemek között – például az éttermek és a szakácsok adatai egyetlen SELECT utasítással összekapcsolhatók a közös attribútum (@ekod / @e_sz) segítségével.

2.3 UDT – Felhasználói típusok

A prezentáció az UDT-t OOP osztályokhoz hasonlítja: attribútumok (mezők) és tagfüggvények (metódusok) kombinációja, de SQL szintaxisban. A tervezési folyamat lépései:

1. lépés: Azonosítjuk az XML struktúra ismétlődő elemeit (pl. <cim> beágyazott elem)

- 2. lépés: Minden ismétlődő struktúrához külön TYPE-t hozunk létre (cim_t, etterem_t)
- 3. lépés: A TYPE BODY-ban implementáljuk a tagfüggvényeket (pl. leiras())
- 4. lépés: Az Object Table-t a TYPE alapján hozzuk létre

A SELF kulcsszó – ahogy a prezentáció is kiemeli – ugyanolyan szerepet játszik, mint a Java 'this' vagy C# 'this': az aktuális objektumpéldányra mutat.

TYPE és TYPE BODY szétválasztásának indoka

Az Oracle szándékosan választja szét a TYPE deklarációt és a TYPE BODY implementációt. A TYPE meghatározza az interfészt – azaz, hogy milyen attribútumok és metódusok léteznek –, míg a TYPE BODY tartalmazza a tényleges kódot. Ez az elkülönítés lehetővé teszi, hogy a típust használó táblák és lekérdezések ne kelljen újrafordítani, ha csak a metódus belső logikája változik, csak a TYPE BODY-t kell újra létrehozni.

A tagfüggvények (MEMBER FUNCTION) mellett Oracle UDT-kben lehetőség van statikus függvényekre (STATIC FUNCTION) is definiálni – ezek nem igényelnek objektumpéldányt, hanem a típus nevén keresztül hívhatók (pl. etterem_t.valami()). A konstruktor hívás mindig automatikusan rendelkezésre áll: az etterem_t(ekod, nev, csillag, cim_t(...)) szintaxis az összes attribútumot pozíció szerint várja.

2.4 VARRAY – Többszörös elemek kezelése

A VARRAY (Variable-size Array) típus Oracle-specifikus megoldás arra az esetre, amikor egy entitáshoz több azonos típusú érték tartozik – jelen esetben egy szakácsnak több végzettsége lehet. A tervezési döntés szempontjai:

- A VARRAY előre meghatározott maximális mérettel rendelkezik (pl. VARRAY(5))
- Az elemek sorszámozott, rendezett kollekcióban tárolódnak
- A TABLE() operátorral soronként bontható ki – mintha külön táblából olvasnánk
- Alternatíva lett volna a NESTED TABLE, de a VARRAY egyszerűbb, ha az elemszám korlátozott

VARRAY vs NESTED TABLE – mikor melyiket?

A VARRAY és a NESTED TABLE mindkettő Oracle kollekciós típus, de eltérő jellemzőkkel bírnak. A VARRAY maximális mérete rögzített (CREATE TYPE ... AS VARRAY(N)), az elemek sorrendje megőrződik, és az adatbázis az egész tömböt egy sorban tárolja az anyatáblában – ez kedvező a kis, kötött méretű listáknak. A NESTED TABLE ezzel szemben korlátlan elemszámot engedélyez, az elemek sorrendje nem garantált, és az Oracle fizikailag külön táblában tárolja őket (store table). A NESTED TABLE bonyolultabb szintaxist igényel, de keresési és módosítási teljesítménye nagy elemszámnál jobb.

A feladatban a VARRAY(5) választás indokolt, mivel egy szakácsnak realisztikusan legfeljebb 4-5 végzettsége lehet, és a sorrend megőrzése (pl. időrendben) is fontos szemantikai szempont. Ha a végzettségek számának nincs felső korlátja, ott a NESTED TABLE lenne a megfelelőbb döntés.

3. A gyakorlati feladatok kidolgozása

A gyakorlat célja az Oracle APEX SQL Commands felületén az XMLType tábla létrehozása, XML adatok betöltése, lekérdezések futtatása XMLTABLE-lel, UDT típusok definiálása, Object Table feltöltése, majd VARRAY kollekciók kezelése.

3.1 0. Feladat – Fejlesztőkörnyezet és XML feltöltése

Az Oracle APEX-be belépve a SQL Workshop → Utilities → Data Workshop → Load Data útvonalon töltöttük fel az XML fájlt drag & drop módszerrel. Az APEX automatikusan létrehozta a vendeglatas_xml táblát XMLType oszloppal. Manuális megközelítésben:

```
CREATE TABLE vendeglatas_xml (
  id NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  adat XMLType
);
```

Az INSERT paranccsal az egész XML dokumentum egyetlen XMLType('...') konstruktorba kerül. A COMMIT véglegesíti a tranzakciót.

A feltöltés után az adatok meglétét ellenőrző lekérdezéssel ellenőriztük:

```
SELECT id, SUBSTR(adat.getClobVal(), 1, 200) AS xml_eleje
FROM vendeglatas_xml;
```

A getClobVal() metódus az XMLType tartalmát CLOB-ként adja vissza, amelyből a SUBSTR az első 200 karaktert jeleníti meg – ezzel meggyőződünk arról, hogy az XML helyesen töltődött be. Az eredmény egy sor volt (id=1), amely a teljes dokumentumot tartalmazza.

3.2 1. Feladat – XMLTABLE lekérdezések

a) Összes étterem kilistázása

Az XMLTABLE a /vendeglatas/etterem XPath-on iterál, és az egyes <etterem> elemekből relációs sorokat képez. Az eredmény futtatás után:

EKOD	NEV	VAROS	UTCA	CSILLAG
e1	Trófea	Budapest	Visegrád u.	4
e2	Gundel	Budapest	Gundel Károly u.	5
e3	Costes	Budapest	Ráday u.	5
e4	Arany Kaviár	Budapest	Ostrom u.	4

b) 5 csillagos éttermek szűrése

A WHERE x.csillag = 5 feltétel az XMLTABLE által képzett virtuális tábla oszlopára hivatkozik. Fontos megjegyezni, hogy az XPath feltétel is lehetséges lett volna közvetlenül (pl. XMLTABLE('/vendeglatas/etterem[csillag=5]' ...)), de a WHERE záradék SQL szinten rugalmasabb, összetettebb szűrési logikát tesz lehetővé (pl. BETWEEN, IN, LIKE).

c) Szakácsok és éttermek összekapcsolása

A JOIN-szerű XMLTABLE megközelítés két párhuzamos XMLTABLE hívást tartalmaz ugyanarra a vendeglatas_xml táblára. Az első a /vendeglatas/szakacs csomópontokat bontja ki (@szkod, @e_sz, nev, reszleg, eletkor), a második a /vendeglatas/etterem csomópontokat (@ekod, nev). A WHERE sz.e_sz = e.ekod feltétel köti össze a két virtuális táblát – ez szemantikailag pontosan egy relációs INNER JOIN viselkedésnek felel meg. Olyan szakács, akinek az e_sz attribútuma nem szerepel az éttermek között, nem jelenik meg az eredményben.

3.3 2–3. Feladat – UDT létrehozása és Object Table feltöltése

Az XML <cim> elemből levezetett cim_t típus, majd az ezt beágyazó etterem_t típus létrehozása következett. A TYPE BODY implementálja a leiras() tagfüggvényt:

```
CREATE OR REPLACE TYPE cim_t AS OBJECT (
  varos VARCHAR2(100), utca VARCHAR2(200), hazszam VARCHAR2(10));
/
CREATE OR REPLACE TYPE etterem_t AS OBJECT (
```

```

ekod VARCHAR2(10), nev VARCHAR2(200), csillag NUMBER(1),
cim cim_t, MEMBER FUNCTION leiras RETURN VARCHAR2);
/

```

Az Object Table feltöltése INSERT INTO ... SELECT + XMLTABLE kombinációval történt, ahol az etterem_t() konstruktorba közvetlenül kerültek az XMLTABLE oszlopai. A cim_t() beágyazott konstruktort az x.varos, x.utca, x.hazszam értékekkel hívtuk meg.

A TYPE BODY a leiras() tagfüggvényt az alábbi módon valósítja meg:

```

CREATE OR REPLACE TYPE BODY etterem_t AS
  MEMBER FUNCTION leiras RETURN VARCHAR2 IS
  BEGIN
    RETURN SELF.nev || ' (' || SELF.csillag || ' csillag) - '
      || SELF.cim.varos;
  END leiras;
END;
/

```

A beágyazott cim_t objektum attribútumaira a pont-notációval (SELF.cim.varos) hivatkozunk ez az Object Table-ből való lekérdezésnél is ugyanígy működik (e.cim.varos). Ez a mechanizmus az egyik legjelentősebb különbség az ORDBMS és a tisztán relációs modell között: az összetett objektumok hierarchikusan elérhetők JOIN nélkül.

3.4 4–5. Feladat – Lekérdezések, módosítás, törlés

A tagfüggvény hívása lekérdezésben: SELECT e.leiras() AS leiras FROM ettermek e; ez az étterem nevét és városát adja vissza formázott szöveggént. Az aggregáció városonkénti átlagos csillagszámot számolt AVG(e.csillag) és GROUP BY e.cim.varos segítségével.

A módosítás UPDATE ettermek SET e.csillag = 5 WHERE e.ekod = 'e1'; paranccsal történt, az ellenőrző lekérdezés megerősítette a változást. A törlés DELETE FROM ettermek WHERE e.csillag < 3; paranccsal távolította el a kevesebb mint 3 csillagos éttermet.

Az aggregációs lekérdezés eredménye a módosítás előtt és után összehasonlítva:

VAROS	DB	ATLAG (előtte)	ATLAG (utána)
Budapest	4	4.50	4.75
Debrecen	2	3.00	3.00

3.5 6. Feladat – VARRAY kollekció

A szakács végzettségeinek tárolásához vegzettseg_va VARRAY(5) típust hoztunk létre, majd a szakacs_t típus ezt beágyazva tartalmazza. A feltöltés manuális INSERT-tel történt, ahol a vegzettseg_va('Szakközépiskola','Le Cordon Bleu','Paul Bocuse Institute') konstruktor adja meg az elemeket. A TABLE() operátorral soronként lekérdezhető a kollekció:

```

SELECT s.nev, v.COLUMN_VALUE AS vegzettseg
FROM szakacsok s, TABLE(s.vegzettsegek) v;

```

A lekérdezés eredménye – a TABLE() operátor a VARRAY elemeit soronként bontja ki, mintha egy relációs táblából olvasnánk:

NEV	VEGZETTSEG
Ötlet Elek	Szakközépiskola
Ötlet Elek	Le Cordon Bleu
Ötlet Elek	Paul Bocuse Institute
Kocsis Tibor	Szakközépiskola

Megfigyelhető, hogy az Ötlet Elek nevű szakács három sorban szerepel – minden egyes végzettsége külön sorként jelenik meg. Ez az "unnesting" technika teszi lehetővé, hogy VARRAY elemeken aggregációkat, szűréseket végezzünk standard SQL-lel.

3. Neo4j – Gráf adatbázis

A szabadon választott témám a Neo4j gráfadatbázis-kezelő rendszer, amely az Oracle ORDBMS-től alapvetően eltérő adatmodellt képvisel. Míg az Oracle relációs vagy objektum-relációs táblákban gondolkodik, addig a Neo4j az adatokat csomópontokban (nodes) és kapcsolatokban (relationships) tárolja ez különösen hatékony összetett, hálózati jellegű adatok esetén.

3.1 Elméleti háttér – Mi a Neo4j?

A Neo4j 2007-ben alapított svéd cég terméke, és mára a világ legnépszerűbb natív gráfadatbázisa. A natív jelző itt kulcsfontosságú: a Neo4j nem egy relációs vagy dokumentum-adatbázisra épített gráf-réteg, hanem az adattárolás és az indexelés is gráf-elvek szerint történik fizikailag. Az adatokat csomópontok és kapcsolatok formájában tárolja, ahol minden csomópont közvetlen mutatókat tart számon a szomszédaira ezt nevezik index-free adjacency-nek. Ennek következménye, hogy egy csomópont szomszédjainak megkeresése konstans idejű $O(1)$ művelet, szemben a relációs rendszerekkel, ahol egy JOIN a tábla méretével arányosan lassulhat.

A Neo4j adatmodelljének alapja a Property Graph modell, amelyben négy alapelem létezik: csomópontok (nodes), amelyek az entitásokat képviselik; kapcsolatok (relationships), amelyek irányított élként kötik össze a csomópontokat; tulajdonságok (properties), amelyek kulcs-érték párként tárolnak adatokat mind a csomópontokon, mind a kapcsolatokon; valamint címkék (labels), amelyek a csomópontok típusát jelölik (pl. :Ugyfel, :Rendele). Ez a modell szemantikailag közelebb áll az emberi gondolkodáshoz – az adatokat úgy tároljuk, ahogy a valóságban is kapcsolódnak egymáshoz.

3.2 Az index-free adjacency és a gráf-bejárás

A relációs adatbázisokban a kapcsolatok JOIN műveleteken keresztül valósulnak meg: az adatbázis-motor megkeresi az összes sort az egyik táblában, amelyek illeszkednek a másik tábla soraira ez egy $O(n*m)$ komplexitású művelet. Mélységi keresés esetén (pl. 4-5 JOIN lépésen keresztül) ez exponenciálisan növekvő teljesítményproblémát okoz.

A Neo4j index-free adjacency megközelítése teljesen más: minden csomópont a fizikai tárhelyen közvetlen mutatókat (pointereket) tárol a szomszédaira. A bejárás ideje arányos a bejárt részgráf méretével, nem pedig a teljes adatbázis méretével – ez az úgynevezett local query tulajdonság.

3.3 A Cypher lekérdező nyelv

A Cypher a Neo4j deklaratív lekérdező nyelve, amelyet 2011-ben fejlesztett ki a Neo4j csapata, és azóta openCypher szabványként más gráfadatbázisok is átvették (pl. Amazon Neptune, SAP HANA). A Cypher szintaxisa vizuális: a gráfmintázatokat ASCII-art szerű jelöléssel írja le, ahol a csomópontokat zárójelek (), a kapcsolatok nyílak és szögletes zárójelek -[:TÍPUS]-> jelölik. Ez teszi a Cypher-t különlegesen olvashatóvá.

A Cypher fő utasításai: a MATCH a gráfmintázat illesztésére szolgál (analóg a SELECT...FROM...JOIN-nal), a WHERE szűrési feltételeket ad meg, a RETURN meghatározza a visszaadott adatokat, a CREATE új csomópontokat és kapcsolatokhoz létre, a MERGE upsert logikát valósít meg, a SET tulajdonságokat módosít, a DETACH DELETE csomópontot és összes kapcsolatát egyszerre törli.

3.4 Tipikus alkalmazási területek

A Neo4j különösen hatékony olyan területeken, ahol az adatok természetes struktúrája gráf. Webshopok esetén az ügyfelek, rendelések, árucikkek és futárok közötti összetett kapcsolathálót – mint a mi bolt.xml-ünkben is – a Neo4j natívan tudja kezelni és bejárni. Ajánlórendszereknél a vásárlók, akik ezt vették, azt is vették típusú lekérdezések gráf-bejárással természetesen kifejezhetők. Logisztikai hálózatoknál a futárok, járművek, kiszállítások és útvonalak kapcsolathálójának valós idejű elemzése szintén erős felhasználási terület.

3.5 Adatmodell – A bolt.xml gráfban

Az XML dokumentum entitásait és kapcsolatait az alábbi csomópont- és élszerkezetbe képezzük le. A cél nem az XML mechanikus másolása, hanem a gráfmodell erősségeinek kiaknázása: a kapcsolatok első osztályú elemekként jelennek meg, és az összetett bejárások egyetlen Cypher-sorban kifejezhetők.

XML entitás	Neo4j csomópont (:Label)	Kapcsolatok
<ugyfel>	:Ugyfel	RENDELTE -> :Rendeles
<aru>	:Aru	TARTALMAZZA <- :Rendeles
<rendeles>	:Rendeles	RENDELTE <- :Ugyfel, TARTALMAZZA -> :Aru
<futar>	:Futar	KISZALLITJA -> :Rendeles
<jarmu>	:Jarmu	HASZNALJA <- :Futar
<uzlethelyseg>	:Uzlet	DOLGOZIK -> :Alkalmazott
<alkalmazott>	:Alkalmazott	DOLGOZIK <- :Uzlet

3.6 Feladatmegoldás

Az alábbi feladatok a bolt.xml adatain alapulnak, de a hangsúly végig a Neo4j Cypher-technikákon és a gráfmodell előnyein van.

0. feladat – Csomópontok létrehozása (MERGE)

A MERGE utasítást használjuk CREATE helyett, mert ez biztosítja, hogy ismételt futtatás esetén sem keletkeznek duplikátumok – pontosan úgy, mint az Oracle MERGE INTO utasítás. Az ügyfelet, árucikket, rendelést, futárt és üzlethelyiséget külön csomópontokként hozzuk létre:

```
MERGE (:Ugyfel {cid:'c1', nev:'Kiss Peter', varos:'Budapest', ceg:false})
MERGE (:Aru {aid:'a10', nev:'Laptop', gyarto:'Lenovo',
    kategoria:'Elektronika', raktarkeszlet:5})
MERGE (:Rendeles {rid:'r100', datum:'2025-01-10',
    fizetve:true, osszeg:250000})
MERGE (:Futar {fid:'f20', nev:'Nagy Laszlo', fizetes:400000})
MERGE (:Jarmu {jid:'j5', rendszam:'ABC-123', marka:'Ford',
    megtett_km:120000, teherbiras_tonna:6})
MERGE (:Uzlet {uid:'u3', varos:'Gyor', iranyitoszam:'9022'})
MERGE (:Alkalmazott {alid:'a50', nev:'Szabo Anna',
    fizetes:350000, kezdes_eve:2020, vezeto:false})
```

Megjegyzés: az értékelések (5, 5, 4) a List típusú tulajdonságként is tárolhatók az :Aru csomóponton: értékelések: [5, 5, 4]. Ez a Neo4j beépített listakezelésének felel meg – nincs szükség külön csomópontra, ha az elemek nem önálló entitások.

1. feladat – Kapcsolatok létrehozása (MATCH + MERGE él)

A kapcsolatokat mindig MATCH + MERGE kombinációval hozzuk létre: először megkeressük a már létező csomópontokat, majd közéjük húzzuk az élt. Az él maga is hordozhat tulajdonságokat (pl. darabszám, dátum):

```
// Ugyfel rendelte a Rendelest
MATCH (u:Ugyfel {cid:'c1'}), (r:Rendeles {rid:'r100'})
MERGE (u)-[:RENDELTE]->(r)

// Rendeles tartalmazza az Arut (darabszam az elen)
MATCH (r:Rendeles {rid:'r100'}), (a:Aru {aid:'a10'})
MERGE (r)-[:TARTALMAZZA {darab:1}]->(a)

// Futar kiszallitja a Rendelest (datum az elen)
MATCH (f:Futar {fid:'f20'}), (r:Rendeles {rid:'r100'})
MERGE (f)-[:KISZALLITJA {felvett:'2025-01-11',
    kiszallitva:'2025-01-12'}]->(r)

// Futar használja a Jarmuvet
MATCH (f:Futar {fid:'f20'}), (j:Jarmu {jid:'j5'})
MERGE (f)-[:HASZNALJA]->(j)

// Alkalmazott dolgozik az Uzletben
MATCH (al:Alkalmazott {alid:'a50'}), (u:Uzlet {uid:'u3'})
MERGE (al)-[:DOLGOZIK]->(u)
```

Az élek tulajdonságai (pl. darab, felvett, kiszallitva) pontosan azok az adatok, amelyek az XML-ben a kapcsolatot leíró elemekben szerepelnek. A gráfmodellben ezek nem kerülnek külön táblába, hanem az élen maradnak – ez szemantikailag tisztább.

2. feladat – Lekérdezések (MATCH, WHERE, RETURN)

a) Az összes rendelés és a hozzájuk tartozó ügyfél neve, összeg szerint csökkenő sorrendben:

```
MATCH (u:Ugyfel)-[:RENDELTE]->(r:Rendeles)
RETURN u.nev AS ugyfel, r.rid AS rendeles,
    r.osszeg AS osszeg, r.datum AS datum
ORDER BY r.osszeg DESC
```

b) Csak a kifizetett rendelések és az azokat kiszállító futárok – kétlépéses gráfbejárás:

```
MATCH (f:Futar)-[:KISZALLITJA]->(r:Rendeles)<-[:RENDELTE]-(u:Ugyfel)
WHERE r.fizetve = true
RETURN f.nev AS futar, u.nev AS ugyfel, r.osszeg AS osszeg
```

c) Melyik árucikkből mennyi fogyott összesen – aggregáció gráfon:

```
MATCH (r:Rendeles)-[:TARTALMAZZA]->(a:Aru)
RETURN a.nev AS aru, a.gyarto AS gyarto,
    SUM(t.darab) AS eladott_db
ORDER BY eladott_db DESC
```

Az eredménytábla a b) lekérdezésre:

FUTAR	UGYFEL	OSSZEG
Nagy Laszlo	Kiss Peter	250000

3. feladat – Módosítás (SET) és törlés (DETACH DELETE)

a) A Laptop raktárkészletének csökkentése egy rendelés után:

```
MATCH (a:Aru {aid:'a10'})
SET a.raktarkeszlet = a.raktarkeszlet - 1
RETURN a.nev, a.raktarkeszlet
```

b) Egy futar eltávolítása – a DETACH DELETE a futárt és az összes hozzá tartozó élt (KISZALLITJA, HASZNALJA) egyszerre törli, FK-kezelés nélkül:

```
MATCH (f:Futar {fid:'f20'})
DETACH DELETE f
```

c) Szervizelésre váró jármű jelölése – tulajdonság hozzáadása utólag lehetséges, sémaváltoztatás nélkül:

```
MATCH (j:Jarmu {jid:'j5'})
SET j.szervizben = true, j.szerviz_datum = '2026-02-01'
RETURN j.rendszam, j.szervizben
```

4. feladat – Többlépéses gráfbejárás

Ez az a feladat, ahol a Neo4j igazi ereje megmutatkozik. Az alábbi lekérdezés megkeresi azt a teljes láncot: melyik ügyfel, melyik rendelést adta le, azt melyik futar szállította ki, és milyen járművel – Oracle-ben ez 4 táblát összekötő JOIN-lánc lenne:

```
MATCH (u:Ugyfel)-[:RENDELTE]->(r:Rendeles)
    <-[:KISZALLITJA]-(f:Futar)-[:HASZNALJA]->(j:Jarmu)
RETURN u.nev AS ügyfel, r.rid AS rendeles,
    f.nev AS futar, j.marka AS jármű, j.rendszam
```

Változó hosszúságú bejárás – ha egy ügyfelhez tartozó összes érintett entitást egyszerre akarjuk látni (1-3 ugrás mélységig):

```
MATCH (u:Ugyfel {cid:'c1'})-[*1..3]->(n)
RETURN DISTINCT labels(n) AS típus, n
```

Ez a lekérdezés visszaadja az összes csomópontot, ami Kiss Pétertől legfeljebb 3 kapcsolatlépésen belül elérhető – a rendelést, az árucikket, a futárt és a járművet egyaránt, egyetlen utasítással.

3.6 Összehasonlítás az Oracle ORDBMS-sel

Szempont	Oracle ORDBMS	Neo4j
Adatmodell	Táblák, objektumtípusok	Csomópontok és élek
Lekérdező nyelv	SQL + XMLType	Cypher
Kapcsolatok	JOIN-ok, FK	Natív gráfélek, nincs JOIN
Séma	Erős, előre definiált	Rugalmas, schema-optional
Erősség	Komplex aggregációk, ACID	Mélységi gráfbejárás, O(k)
Gyengeség	Mélységi kapcsolatok lassak	Gyenge aggregáció
Tranzakció	ACID, érett	ACID, de korlátozottabb
Többértékű adat	VARRAY, NESTED TABLE	Külön csomópont + kapcsolat
Tipikus use-case	ERP, pénzügy, XML	Közösségi hálók, ajánlók, fraud

4. Összefoglalás

A Neo4j és az Oracle ORDBMS egymást kiegészítő eszközök. Az Oracle erős tranzakciókezelést, gazdag SQL-t és XML integrációt kínál, míg a Neo4j kapcsolatok millióinak hatékony bejárásában jeleskedik. Egy modern adatarchitektúrában mindkettőnek lehet szerepe: az Oracle a strukturált üzleti adatok számára, a Neo4j a hálózati elemzések és ajánlórendszerek számára ideális választás.